Base Julios GOD

Contents

1 Data structures

1.1 Segment tree

```
#define M ((l + r) >> 1)
```

1.2 Segment tree - Range update, point query

```
// Range update - point query
```

1.3 Segment tree - Lazy propagation

```
#define M ((l + r) >> 1)
```

1.4 Segment tree - Persistence

```
#define M ((l + r) >> 1)
```

1.5 Fenwick tree

```
struct FT{
```

1.6 DSU

```
struct DSU{
```

1.7 SQRT decomposition

```
struct SQ{
```

1.8 Trie

```
struct Node{
```

1.9 Trie XOR

```
struct Node{
```

1.10 Map custom hash

```
#include <ext/pb_ds/assoc_container.hpp>
```

1.11 Ordered set

```
#include<ext/pb_ds/assoc_container.hpp>
```

1.12 Merge Sort Tree

```
#define M ((l + r) >> 1)
```

1.13 Wavelet Tree

```
// indexed in 1
```

1.14 Monotonic stack

```
int main() {
```

1.15 Matrix operations

```
struct Matrix {
```

1.16 Heavy Light Decomposition

```
int sz[N], in[N], out[N], timer, head[N], parent[N];
```

1.17 Convex Hull Trick

```
typedef ll tc;
```

1.18 Dynamic Convex Hull Trick

```
using ll = long long;
```

2 Graphs

2.1 Euler tour

```
vector<int> in(n+1), out(n+1), euler(n+1);
```

2.2 Find bridges

```
int in[N], low[N], timer;
```

2.3 Dijkstra

```
void dijkstra(int start, int n) {
```

2.4 Binary Lifting LCA

```
int depth[N], up[N][LOG];
```

2.5 Centroid decomposition

```
void dfs(int u, int p) {
```

2.6 Edmonds karp

```
const int N = 1e3+5;
```

2.7 Dinics

```
struct Dinic {
```

2.8 Bipartite check

```
int color[N];
```

2.9 Has cycle?

```
int color[N];
```

2.10 Kuhn - matching

```
vector<int> adj[N]; // [0,n)->[0,m)
```

2.11 Hopcroft Karp - matching

```
vector<int> g[MAXN]; // [0,n)->[0,m)
```

2.12 Min cost - max flow

```
typedef ll tf;
```

2.13 2sat

```
struct two_sat {
```

3 Math

3.1 Binpow

```
ll binpow(ll a, ll b, ll m) {
```

3.2 Modular inverse

```
tuple<int, int, int> extendedGCD(int mod, int a) {
```

3.3 Linear Sieve

```
const int N = 1e7+5;  // N: range to get primes
```

3.4 Factorials

```
void pre() {
```

3.5 Prime factorization

```
set<int> factors;
```

3.6 Divisors

```
vector<int> div;
```

3.7 Identities

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1}$$
$$C_n = \frac{1}{n+1}\binom{2n}{n}$$
$$C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$$
$$\sigma(n) = O(\log(\log(n))) \text{ (number of divisors of } n)$$
$$F_{2n+1} = F_n^2 + F_{n+1}^2$$
$$F_{2n} = F_{n+1}^2 - F_{n-1}^2$$
$$\sum_{i=1}^{n} F_i = F_{n+2} - 1$$
$$F_{n+i}F_{n+j} - F_nF_{n+i+j} = (-1)^n F_iF_j$$

(Fermat's little theorem) $a^p \, mod(p) = a \quad a^{p-1} \, mod(p) = 1 \quad a^{p-2} \, mod(p) = a^{-1}$

(Möbius Inv. Formula) Let $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum d \mid ng(d)\mu\left(\frac{n}{d}\right)$.

4 Strings

4.1 KMP

```
vector<int> prefix_function(const string &s){
```

4.2 Rolling Hash

```
#define rep(i,a,b) for(int i=a;i<b;i++)
```

4.3 LCP using hashing + bs

```
int getLCP(int i, int j, int len) {
```

4.4 Suffix Automaton

```
const int M = 27;
```

4.5 Suffix Array

```
int sa[N], lcp[N];
```

5 Geometry

5.1 Convex Hull

```
const double eps = 1e-9;
```

6 Techniques

6.1 MO's algorithm

```
const int N = 3e5+5, B = 550;
```

6.2 Parallel binary search

```
void parallel_bs() {
```

6.3 Split objects into light and heavy

```
vector<pair<int, int>> adj[N];
```

6.4 Meet in the middle

```
int main() {
```

6.5 Dijkstra on ST

```
vector<pair<int, int>> adj[2*4*N];
```

6.6 Venice set

```
struct VeniceSet {
```

7 Basic techniques

7.1 Difference array

```
// If we only need to get the final result, we can increase l and r+1
```

7.2 Prefix Sum 2D

```
// Compute prefix sum 2D
```

7.3 Random number generator

```
// Secure random seed
```

7.4 Coordinate compression

```
// Push to an array all possible values
```

7.5 Digit DP

```
int dp[11][2][2];
```

7.6 Intersection [L1, R1] and [L2, R2]

```
We need to check whether there is any number common to both ranges:
```